

```

    dbgprint(*(p.left), L, M), dbgprint(*(p.right), M + 1, R);
}

int main() {
    ios::sync_with_stdio(false), cin.tie(0);
    cin >> N;
    string s;
    root = new Node();
    for (int a, b, d, h; cin >> s && s[0] != 'E'; ) {
        if (s[0] == 'I') cin >> a >> b >> d, modify(a, b, d);
        else cin >> h, cout << query(h) << endl;
    }
    delete root;
    return 0;
}
/*
算法分析请参考：《算法竞赛入门经典——训练指南》升级版 3.2.4 节例题 11
注意：如何使用内存动态分配以及释放的技巧来通过本题的空间限制
同类题目：Color it, HDU 6183
*/

```

本节例题列表

本节讲解的例题及其囊括的知识点，如表4-3所示。

表4-3 区间信息维护例题归纳

编 号	题 号	标 题	知 识 点	代 码 作 者
例 4-17	POJ 3928	Ping pong	树状数组；维护逆序对	陈锋
例 4-18	POJ 3368	Frequent Values	RMQ	刘汝佳
例 4-19	UVa 1400	Ray, Pass me the Dishes	线段树；维护区间前后缀	刘汝佳
例 4-20	UVa 11992	Fast Matrix Operations	线段树；区间修改；懒标记传递	刘汝佳
例 4-21	NC 19889	维护序列	线段树；区间修改；乘法操作	陈锋
例 4-22	NC 50456	花神游历各国	线段树；区间修改；根号操作	陈锋
例 4-23	SPOJ NKMOU	Mountains	线段树；区间修改；动态开点	陈锋

4.3 排序二叉树

本节主要介绍基于排序二叉树数据结构（包括STL中map、set、Treap以及Splay）的经典题目及其代码实现。

例 4-24 【map 维护点集；单调性】优势人群（Efficient Solutions, UVa 11020）

有 n 个人（ $0 \leq n \leq 15\,000$ ），每个人有两个属性 x 和 y （不超过 10^9 的非负整数）。如果对于一个人 $P(x, y)$ ，不存在另外一个人 (x', y') ，使得 $x' < x$ ， $y' \leq y$ ，或者 $x' \leq x$ ， $y' < y$ ，我们就说 P 是有优势的。每次给出一个人的信息，要求输出只考虑当前已获得信息的前提下，多少人是有优势的。

【代码实现】

```

// 刘汝佳
#include <bits/stdc++.h>
using namespace std;
typedef long long LL;
struct Point {
    int x, y;
    bool operator<(const Point& p2) const {
        if (x != p2.x) return x < p2.x;
        return y < p2.y;
    }
};

int main() {
    int T;
    scanf("%d", &T);
    for (int n, x, y, t = 1; t <= T; t++) {
        scanf("%d", &n);
        if (t > 1) puts("");
        printf("Case #%d:\n", t);
        multiset<Point> s;
        for (int i = 0, x, y; i < n; i++) {
            scanf("%d%d", &x, &y);
            Point p = {x, y};
            multiset<Point>::iterator it = s.lower_bound(p);
            if (it == s.begin() || (--it)->y > p.y) {
                s.insert(p), it = s.upper_bound(p);
                while (it != s.end() && it->y >= p.y) s.erase(it++);
            }
            printf("%lu\n", s.size());
        }
        return 0;
    }
}
/*
算法分析请参考：《算法竞赛入门经典——训练指南》升级版 3.6.2 节例题 29
注意：在 multiset 中删除元素之前必须判断是否为 end()，也就是说不存在的元素
同类题目：Cut the Sequence, POJ 3017
*/

```

例4-25 【名次树；并查集；时光倒流】图询问（Graph and Queries, 天津2010, HDU 3726）

有一张包含 n 个结点、 m 条边的无向图（ $1 \leq n \leq 20\,000$ ， $0 \leq m \leq 60\,000$ ），每个结点都有一个不超过 10^6 的整数权值。你的

任务是执行一系列指令操作。操作分为3种，如表4-4所示。

表4-4 可执行的3种指令操作

操 作	备 注
D X ($1 \leq X \leq m$)	删除 ID 为 X 的边。输入保证每条边至多被删除一次
Q $X k$ ($1 \leq X \leq n$)	计算与结点 X 连通的结点中（包括 X 本身），第 k 大的权值。如果不存在，输出 0
C $X V$ ($1 \leq X \leq n$)	把结点 X 的权值改为 V

操作序列的结束标志为单个字母E。结点编号为 $1 \sim n$ ，边编号为 $1 \sim m$ 。

输入第一行为两个整数 n 和 m ；以下 n 行每行有一个绝对值不超过 10^6 的整数，即各结点的初始权值；以下 m 行每行有两个整数，即一条边的两个端点；接下来是各条指令，以单个字母E结尾。保证Q指令和C指令均不超过200 000条。输入结束标志为 $n=m=0$ 。

对于每组数据，输出所有Q指令的计算结果的平均值，精确到小数点后6位。

【代码实现】

```

// 刘汝佳
#include <bits/stdc++.h>
using namespace std;
typedef long long LL;

struct Node {
    Node* ch[2]; // 左右子树
    int r, v, s; // 随机优先级, 值, 结点总数
    Node(int v) : v(v), r(rand()), s(1) { ch[0] = ch[1] = NULL; }
    int cmp(int x) const {
        if (x == v) return -1;
        return x < v ? 0 : 1;
    }
    void maintain() {
        s = 1;
        if (ch[0]) s += ch[0]->s;
        if (ch[1]) s += ch[1]->s;
    }
};

void rotate(Node*& o, int d) {
    Node* k = o->ch[d ^ 1];
    o->ch[d ^ 1] = k->ch[d];
    k->ch[d] = o;
    o->maintain(), k->maintain();
    o = k;
}

void insert(Node*& o, int x) {
    if (o == NULL)
        o = new Node(x);
    else {
        int d = (x < o->v ? 0 : 1); // 不要用 cmp 函数, 因为可能会有相同结点
        insert(o->ch[d], x);
    }
}

```

```

    if (o->ch[d]->r > o->r) rotate(o, d ^ 1);
}
o->maintain();
}

void remove(Node*& o, int x) {
    int d = o->cmp(x);
    if (d == -1) {
        Node* u = o;
        if (o->ch[0] != NULL && o->ch[1] != NULL) {
            int d2 = (o->ch[0]->r > o->ch[1]->r ? 1 : 0);
            rotate(o, d2), remove(o->ch[d2], x);
        } else {
            if (o->ch[0] == NULL)
                o = o->ch[1];
            else
                o = o->ch[0];
            delete u;
        }
    } else
        remove(o->ch[d], x);
    if (o) o->maintain();
}

const int maxc = 500000 + 4;
struct Command {
    char type;
    int x, p; // 根据 type, p 代表 k 或者 v
} Cmds[maxc];
const int maxn = 20000 + 4, maxm = 60000 + 4;
int n, m, weight[maxn], from[maxm], to[maxm], removed[maxm];
int pa[maxn]; // 并查集相关
int findset(int x) { return pa[x] != x ? pa[x] = findset(pa[x]) : x; }
Node* root[maxn]; // Treap, 名次树相关

int kth(Node* o, int k) { // 第 k 大值
    if (o == NULL || k <= 0 || k > o->s) return 0;
    int s = (o->ch[1] == NULL ? 0 : o->ch[1]->s);
    if (k == s + 1) return o->v;
    if (k <= s) return kth(o->ch[1], k);
    return kth(o->ch[0], k - s - 1);
}

void mergeto(Node*& src, Node*& dest) {
    if (src->ch[0]) mergeto(src->ch[0], dest);
    if (src->ch[1]) mergeto(src->ch[1], dest);
    insert(dest, src->v), delete src, src = NULL;
}

```

```

void removetree(Node*& x) {
    if (x->ch[0]) removetree(x->ch[0]);
    if (x->ch[1]) removetree(x->ch[1]);
    delete x, x = NULL;
}

// 主程序相关
void add_edge(int x) {
    int u = findset(from[x]), v = findset(to[x]);
    if (u != v) {
        if (root[u]->s < root[v]->s)
            pa[u] = v, mergeto(root[u], root[v]);
        else
            pa[v] = u, mergeto(root[v], root[u]);
    }
}

int query_cnt;
LL query_tot;
void query(int x, int k) { query_cnt++, query_tot += kth(root[findset(x)], k); }

void change_weight(int x, int v) {
    int u = findset(x);
    remove(root[u], weight[x]), insert(root[u], v), weight[x] = v;
}

int main() {
    for (int kase = 1; scanf("%d%d", &n, &m) == 2 && n; kase++) {
        for (int i = 1; i <= n; i++) scanf("%d", &weight[i]);
        for (int i = 1; i <= m; i++) scanf("%d%d", &from[i], &to[i]);
        memset(removed, 0, sizeof(removed));

        int c = 0; // 读命令
        while (true) {
            char type;
            int x, p = 0, v = 0;
            scanf(" %c", &type);
            if (type == 'E') break;
            scanf("%d", &x);
            if (type == 'D') removed[x] = 1;
            if (type == 'Q') scanf("%d", &p);
            if (type == 'C') scanf("%d", &v), p = weight[x], weight[x] = v;
            Cmds[c++] = (Command){type, x, p};
        }

        // 最终的图
        for (int i = 1; i <= n; i++) {
            pa[i] = i;
            if (root[i] != NULL) removetree(root[i]);
        }
    }
}

```

```

    root[i] = new Node(weight[i]);
}
for (int i = 1; i <= m; i++)
    if (!removed[i]) add_edge(i);

// 反向操作
query_tot = query_cnt = 0;
for (int i = c - 1; i >= 0; i--) {
    if (Cmds[i].type == 'D') add_edge(Cmds[i].x);
    if (Cmds[i].type == 'Q') query(Cmds[i].x, Cmds[i].p);
    if (Cmds[i].type == 'C') change_weight(Cmds[i].x, Cmds[i].p);
}
printf("Case %d: %.6lf\n", kase, query_tot / (double)query_cnt);
}
return 0;
}
/*
算法分析请参考：《算法竞赛入门经典——训练指南》升级版 3.6.2 节例题 30
注意：本题中对于引用指针类型的操作
同类题目：Black Box, POJ 1442
*/

```

例4-26 【伸展树/splay; Treap; 可分裂合并的序列】排列变换 (Permutation Transformer, UVa 11922)

你的任务是根据 m 条指令改变排列 $\{1, 2, 3, \dots, n\}$ ($1 \leq n, m \leq 100\,000$)。每条指令 (a, b) 表示取出第 $a \sim b$ 个元素，翻转后添加到排列的尾部。输出 n 行，即最终排列。

【代码实现（基于Splay）】


```

// 刘汝佳
#include <algorithm>
#include <cstdio>
#include <vector>
using namespace std;

struct Node {
    Node* ch[2];
    int s, flip, v;
    int cmp(int k) const {
        int d = k - ch[0]->s;
        if (d == 1) return -1;
        return d <= 0 ? 0 : 1;
    }
}

void maintain() { s = ch[0]->s + ch[1]->s + 1; }
void pushdown() {
    if (flip) {
        flip = 0;
        swap(ch[0], ch[1]);
        ch[0]->flip = !ch[0]->flip;
        ch[1]->flip = !ch[1]->flip;
    }
}

```

```

    }
}
};

Node* null = new Node();

void rotate(Node*& o, int d) {
    Node* k = o->ch[d ^ 1];
    o->ch[d ^ 1] = k->ch[d];
    k->ch[d] = o;
    o->maintain();
    k->maintain();
    o = k;
}

void splay(Node*& o, int k) {
    // 找到序列的左数第 k 个元素并伸展到根结点
    o->pushdown();
    int d = o->cmp(k);
    // 看看第 k 个元素在整个树中的位置
    if (d == 1) k -= o->ch[0]->s + 1;
    // 第 k 个元素在 o 的右子树中
    if (d == -1) return;
    // 已经在根上了
    Node* p = o->ch[d];
    // 第 k 个元素所在的子树
    p->pushdown();
    int d2 = p->cmp(k);
    // 第 k 个元素在 p 的左子树?→d2
    int k2 = (d2 == 0 ? k : k - p->ch[0]->s - 1);
    // 在树中的排名
    if (d2 != -1) {
        // 不是子树的根, 伸展到 p
        splay(p->ch[d2], k2);
        // 伸展到 p 的子树根, 下面旋转到 p
        if (d == d2)
            rotate(o, d ^ 1);
        // 一条直线
        else
            rotate(o->ch[d], d);
        // 不是一条直线
    }
    rotate(o, d ^ 1);
    // 从 p 旋转到 o
}

// 合并 left 和 right. 假定 left 的所有元素比 right 小. right 可以是 null, 但 left 不可以
Node* merge(Node* left, Node* right) {
    splay(left, left->s);
    left->ch[1] = right;
    left->maintain();
    return left;
}

// 把 o 的前 k 小结点放入 left, 其余的放入 right. 1 ≤ k ≤ o->s. 若 k=o->s, right=null
void split(Node* o, int k, Node*& left, Node*& right) {
    splay(o, k);
    left = o;
    right = o->ch[1];
    o->ch[1] = null;
    left->maintain();
}

```

```

}

const int NN = 100000 + 10;
struct SplaySequence {
    int n;
    Node seq[NN];
    Node* root;

    Node* build(int sz) {
        if (!sz) return null;
        Node* L = build(sz / 2);
        Node* o = &seq[++n];
        o->v = n; // 结点编号
        o->ch[0] = L;
        o->ch[1] = build(sz - sz / 2 - 1);
        o->flip = o->s = 0;
        o->maintain();
        return o;
    }

    void init(int sz) { n = 0, null->s = 0, root = build(sz); }
};

vector<int> ans;
void print(Node* o) {
    if (o == null) return;
    o->pushdown();
    print(o->ch[0]);
    ans.push_back(o->v);
    print(o->ch[1]);
}

void debug(Node* o) {
    if (o == null) return;
    o->pushdown();
    debug(o->ch[0]);
    printf("%d ", o->v - 1);
    debug(o->ch[1]);
}

SplaySequence ss;
int main() {
    int n, m;
    scanf("%d%d", &n, &m);
    ss.init(n + 1); // 最前面有一个虚拟结点
    for (int i = 0, a, b; i < m; i++) {
        scanf("%d%d", &a, &b);
        Node *left, *mid, *right, *o;
        split(ss.root, a, left, o); // 如无虚拟结点, a 将改成 a-1, 违反 split 的限制
    }
}

```

```

    split(o, b - a + 1, mid, right);
    mid->flip ^= 1;
    ss.root = merge(merge(left, right), mid);
}

print(ss.root);
for (size_t i = 1; i < ans.size(); i++)
    printf("%d\n", ans[i] - 1);    // 结点编号减1才是本题的元素值

return 0;
}
/*
算法分析请参考：《算法竞赛入门经典——训练指南》升级版 3.6.3 节例题 31
*/

```

【代码实现（基于Treap）】

```

// 魏子豪/陈锋
#include <bits/stdc++.h>
using namespace std;

struct Node {
    // 左右子树, 子树体积, 随机优先级, 权值, 翻转标志
    int left, right, size, pr, val, rev;
};

template <size_t SZ>
struct FuncTreap {
    Node _B[SZ];
    size_t sz, root;
    void init() { sz = 0, root = 0; }
    int new_node(int _v) {
        assert(sz + 1 < SZ);
        Node& p = _B[++sz];
        p.left = p.right = p.rev = 0;
        p.size = 1, p.val = _v, p.pr = rand();
        return sz;
    }

    inline void update(int x) {
        Node& p = _B[x];
        p.size = _B[p.left].size + _B[p.right].size + 1;
    }

    inline void pushdown(int x) {
        Node& p = _B[x];
        if (!p.rev) return;
        swap(p.left, p.right), p.rev = 0;
        _B[p.left].rev ^= 1, _B[p.right].rev ^= 1;
    }
};

```

```

int merge(int x, int y) {
    if (!x || !y) return x + y;
    Node &px = _B[x], &py = _B[y];
    if (px.pr < py.pr) {
        pushdown(x), px.right = merge(px.right, y), update(x);
        return x;
    }
    pushdown(y), py.left = merge(x, py.left), update(y);
    return y;
}

void split(int u, int k, int& x, int& y) {
    if (!u) {
        x = y = 0;
        return;
    }
    pushdown(u);
    Node &p = _B[u], &lp = _B[p.left];
    if (lp.size < k)
        x = u, split(p.right, k - lp.size - 1, p.right, y);
    else
        y = u, split(p.left, k, x, p.left);
    update(u);
}

inline void insert(int v) { root = merge(root, new_node(v)); }

inline void print(int x, char pfx) {
    pushdown(x);
    Node& p = _B[x];
    if (p.left) print(p.left, pfx);
    printf("%d%c", x, pfx);
    if (p.right) print(p.right, pfx);
}

};

const int NN = 1e6;
FuncTreap<NN> T;
int main() {
    int n, m;
    T.init(), srand(time(NULL)), scanf("%d %d", &n, &m);
    T.root = T.new_node(1);
    for (int i = 2; i <= n; i++) T.insert(i);
    for (int i = 0, L, R, rL, rR, rM; i < m; i++) {
        scanf("%d %d", &L, &R);
        T.split(T.root, R, rL, rR), T.split(rL, L - 1, rL, rM);
        T._B[rM].rev ^= 1;
        T.root = T.merge(rL, T.merge(rR, rM));
    }
}

```